

Security Audit

OmniSwap (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	26
Conclusion	27
Our Methodology	28
Disclaimers	30
About Hashlock	31

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The OmniSwap team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Omnipump, by OmniSwap, is a revolutionary platform designed to transform token creation and trading under the "create to earn" initiative. The platform offers users the ability to create their own tokens easily and earn income through an innovative multi-tier referral system.

Project Name: OmniSwap

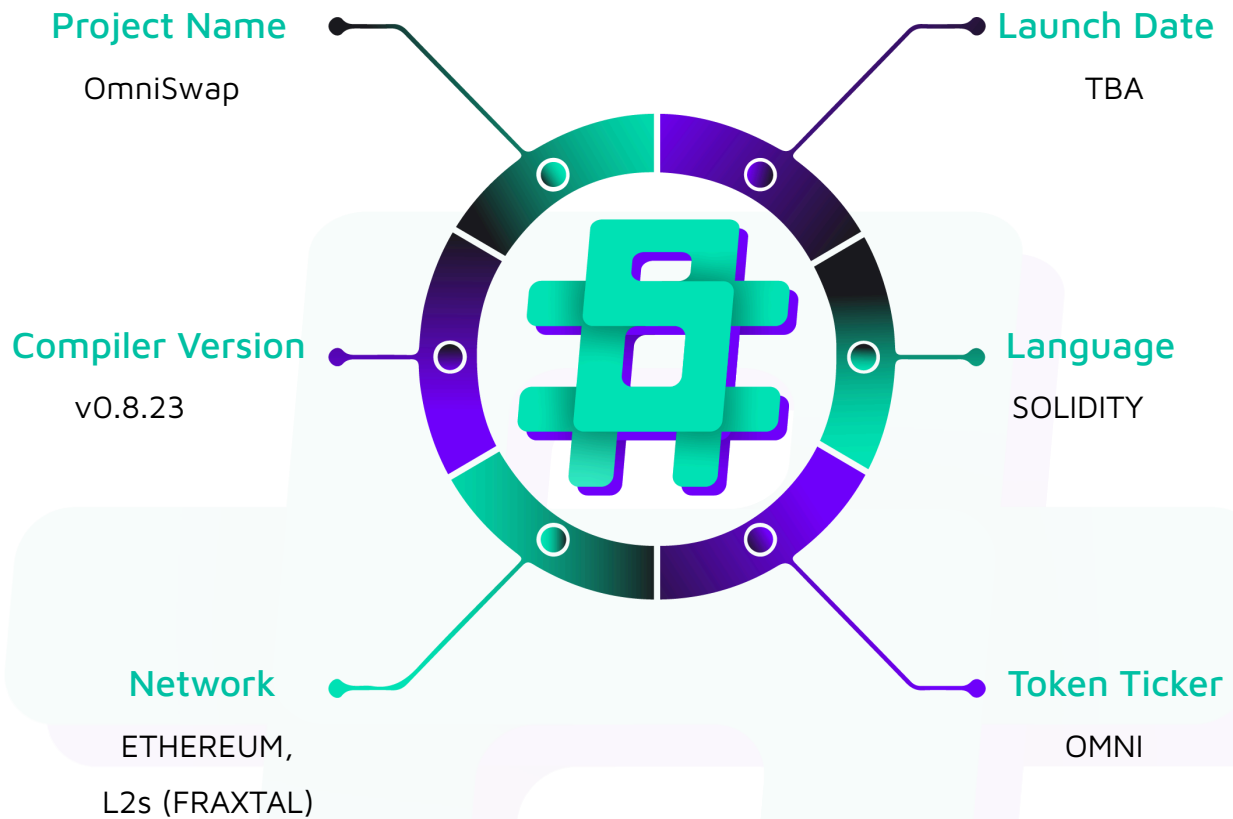
Project Type: Defi, Token

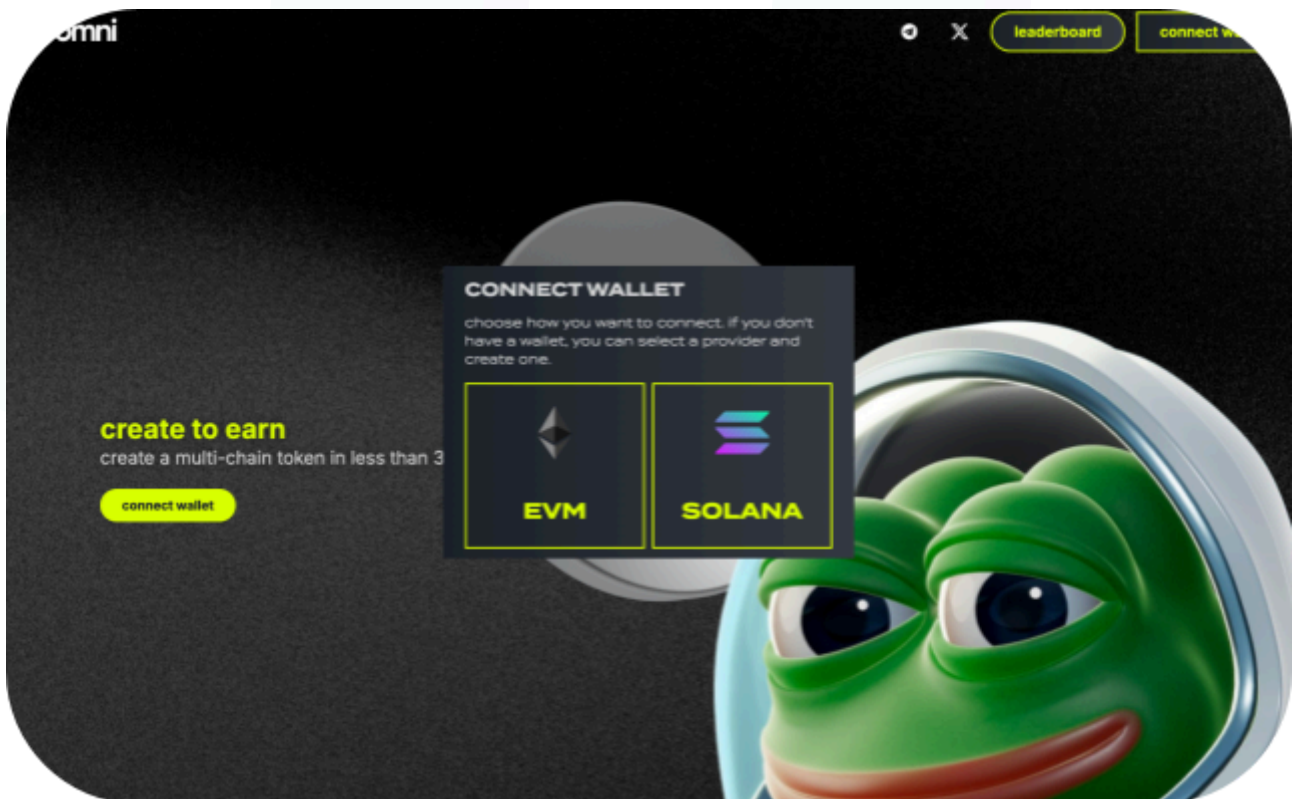
Compiler Version: 0.8.23

Website: <https://omniswap.ai/>

Logo:



Visualised Context:

Project Visuals:

Audit Scope

We at Hashlock audited the solidity code within the OmniSwap project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	OmniSwap Smart Contracts
Platform	Ethereum / Solidity
Audit Date	May, 2025
Contract 1	Splitter.sol
Contract 2	BaseERC20.sol
Contract 3	Constants.sol
Contract 4	ITokenFactory.sol
Contract 5	Curve.sol
Contract 6	ERC20TokenFactory.sol
Contract 7	ICurve.sol
Audited GitHub Commit Hash	271b6e7529536fa8d553fc9500ae53bb41380176
Fix Review GitHub Commit Hash	ed16da02e655b3cf96890f7013c5990142589721

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Medium severity vulnerabilities

4 Low severity vulnerabilities

5 Gas Optimisations

2 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
BaseERC20.sol - Creates the uniswap pair - Allows users to buy and sell ETH/token and create the pool when there is enough liquidity to add to it	Contract achieves this functionality.
ERC20TokenFactory.sol - Allows creators to create token contracts and buy some tokens	Contract achieves this functionality.
Curve.sol - Swapping calculations	Contract achieves this functionality.
Splitter.sol - Distributes treasury	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the OmniSwap project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the OmniSwap project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01]BaseERC20#swapTokensForETH - ERC20TokenFactory#createERC20AndBuy - Lack of slippage protection on swaps can lead to loss of funds for the users

Description

Lack of slippage protection can lead to loss of funds for token creators in `createERC20AndBuy(...)` and users transferring that token after the pool creation in `swapTokensForETH(...)`

Vulnerability Details

Here's the vulnerability in `createERC20AndBuy(...)`:

```
if (remainingForBuy > 0) {  
    BaseERC20(payable(_token)).buy{ value: remainingForBuy }(msg.sender, referrer, 0);  
}
```

`buy(...)` (subcall):

```
function buy(address recipient, address referrer, uint256 minOut) public payable  
whenLiquidityNotAdded nonReentrant() {  
    //...  
    if (amountReceived < minOut) {  
        revert ErrExceedsSlipage();  
    }  
    //...  
}
```

Here's the vulnerability in `swapTokensForETH(...)`:

```
univ2router.swapExactTokensForETHSupportingFeeOnTransferTokens(  
    tokenAmount,  
    0, // @audit no slippage protection  
    path,  
    tokenFactory.treasury(),  
    block.timestamp  
);
```

Impact

Might lead to a loss of funds for the protocol

Recommendation

Add slippage protection for the vulnerable functions cited above

Status

Resolved

[M-02] BaseERC20#getRefAndTransfer - Malicious referrer can revert on referral fee reception

Description

A malicious 2nd or 3rd generation referrer can decide to sacrifice around \$5 worth of token to deny an innocent user a transfer worth around \$10,000 when buying/selling before the creation of the pool.

Vulnerability Details

When a user calls `buy(...)`, `sell(...)`, `buyExactOut(...)`, `sellExactOut(...)`, here's what is happening:

- Tax is set to be 1% of the amount transferred
- `getRefAndTransfer(...)` is called
- `transferToRef(...)` is called to send the reward for the 3 generation referrers (20%) of the tax for the main referrer, and 5% for the others
- If any of the 3 referrers reverts when their fallback function is triggered, the whole tx reverts preventing the user from transferring tokens permanently.

Proof of Concept

This is a detailed attack scenario highlighting the vulnerability:

- User A refers User B
- User B refers User C
- User C creates token0
- Users try to buy/sell this token0 \$10,000
- Tax = 1% amount of token being moved (\$100)
- User A gets 5% of this tax (\$5)
- User A can decide to revert instead of receiving these \$5 and deny the transfer to the users trading token0

Impact

High:DoS, but likelihood is low even though the attack is cheap (\$5 sacrificed to deny \$10k transfer in our scenario), the attacker is still losing \$5 to perform this attack without getting any funds. His sole motivation would be to be malicious to some users.

Recommendation

Implement the pull-over-push pattern

Status

Resolved

Low

[L-01] BaseERC20#receive - Sending tokens directly to the contract through a fallback will always revert

Description

When a user makes an arbitrary call to `BaseERC20` with the intent of triggering the fallback function `receive()`, the call will always revert.

Vulnerability Details

When a user sends ETH to the contract, this fallback is triggered:

```
receive() external payable { // @audit this will always revert in buy() because no eth is
explicitly sent, should send eth as msg.value with call to buy()

    buy(msg.sender, address(0), 0);

}
```

`buy(...)` will revert because `msg.value=0` as it was not set explicitly:

```
function buy(address recipient, address referrer, uint256 minOut) public payable
whenLiquidityNotAdded nonReentrant() {

    //...

    if (msg.value == 0) {

        revert ErrValueMustBeGreaterThanZero(); // @audit reverts here

    }

    //...

}
```

Impact

The function is not working as expected.

Recommendation

Since it currently has no use, you can either:

- Remove it completely
- Fix it by passing a `msg.value` with the call to `buy(...)`

Status

Resolved

[L-02] ERC20TokenFactory#createERC20AndBuy - Might revert unexpectedly and is vulnerable to front-running

Description

`createERC20AndBuy(...)` might revert in case a token creator tries to create a token with an already existing name and symbol without a proper error message. A malicious token creator might also front-run another token creator's call to `createERC20AndBuy(...)` to steal the name and symbol of the other creator's token.

Vulnerability Details

```
bytes memory creationCode = type(BaseERC20).creationCode;

    bytes memory initCode = abi.encodePacked(creationCode, abi.encode(

        wnt,

        univ2router,

        _name,

        _symbol,

        TOTAL_SUPPLY,

        univ2factory,

        curve)

    );

    bytes32 salt = keccak256(abi.encodePacked(block.chainid));
```

```

address _token;

assembly {
    _token := create2(0, add(initCode, 32), mload(initCode), salt)
}

```

Proof of Concept

This is a detailed scenario highlighting the vulnerability:

- Token creator A calls `createERC20AndBuy(...)` successfully to create token0
- Token creator B calls `createERC20AndBuy(...)` with same name and symbol but the call fails, the created token would have the `address(0)`
- The `deposit()` function would revert

This is a detailed attack scenario highlighting the vulnerability:

- Token creator A calls `createERC20AndBuy(...)` to create token0 and tx is pending
- Token creator B sees the pending tx in the mempool and decides to front-run it.
- Token creator B calls `createERC20AndBuy(...)` with same name and symbol, his tx is processed before that of token creator A
- When the tx of token creator A is processed, it fails

Impact

The call would fail with an unknown reason. If it was an arbitrary call, sending tokens directly to the contract instead of this line:

```
BaseERC20(payable(_token)).deposit{value: INITIAL_DEPOSIT}();
```

Then the impact would have been much bigger as the ETH would be sent to the `address(0)`, losing the token creator's funds.

A big impact would be a token creator getting "front-ran" and being prevented from creating a token with the name and symbol he had in mind. He can still choose another name and symbol.

Recommendation

Check if `_token == address(0)` and revert with an adequate error message if it is.

Status

Resolved

[L-03] Constants - Comment description does not align with the value of the constant

Description

The Constants.sol library has a constant that is set to the wrong value according to the comment.

```
uint256 public constant INITIAL_VIRTUAL_TOKEN_RESERVE = 450_660_000_000 ether; // 450.66B

uint256 public constant INITIAL_VIRTUAL_NATIVE_RESERVE = 1.5 ether; // 1.5 ether

uint256 public constant VIRTUAL_TOKEN_RESERVE_WHEN_ADD_LIQUIDITY = 117_550_000_000 ether; // 117.55 B

uint256 public constant TOTAL_TOKEN_FOR_SELL = INITIAL_VIRTUAL_TOKEN_RESERVE - VIRTUAL_TOKEN_RESERVE_WHEN_ADD_LIQUIDITY; // 117.55 B // @audit do you want it to be 117.55 B because now it is 333.11 B
```

Recommendation

Fix the comment or `TOTAL_TOKEN_FOR_SELL`

Status

Resolved

[L-04] BaseERC20#addLiquidity - Checks to allow pool creation are too strict and can make it difficult to add liquidity especially with malicious actors

Description

The checks to allow a user to add liquidity to the pair of tokens and thus create the pool is very strict and can make the process of creating a pool challenging especially with the presence of malicious actors.

Vulnerability details

That's the internal function called at the end of these 4 functions:

- `buy(...)`
- `sell(...)`
- `buyExactOut(...)`
- `sellExactOut(...)`

```
function callAddLiquidityIfShould() internal {
    if (virtualReserves[1] == Constants.VIRTUAL_TOKEN_RESERVE_WHEN_ADD_LIQUIDITY) {
// @audit-issue check is too strict use >= instead
        addLiquidity();
    }
}
```

Here's the `addLiquidity()` function check:

```
function addLiquidity() public whenLiquidityNotAdded {
    if (virtualReserves[1] != Constants.VIRTUAL_TOKEN_RESERVE_WHEN_ADD_LIQUIDITY) {
// @audit-issue check is too strict use < instead
        revert ErrLiquidityBelowThreshold();
    }
}
```

These checks are too strict and might prevent adding liquidity and creating pools if some malicious users keep on front-running in order to keep:

```
virtualReserves[1] != Constants.VIRTUAL_TOKEN_RESERVE_WHEN_ADD_LIQUIDITY
```

Recommendation

Make the checks less rigid by using operators such as: `<` instead.

Status

Resolved

Gas

[G-01] BaseERC20#_transfer - The function contains duplicate code that should be removed for a better optimization and readability

Description

The `_transfer(...)` function contains duplicate code that should be removed for a better optimization and readability.

Recommendation

Remove this block of code

```
else if ((to != address(this) && from != address(this))) { // @audit this would have
reverted above

    if (!isLiquidityPoolCreated) {

        revert ErrLiquidityNotAdded();

    }

}
```

Status

Acknowledged

[G-02] BaseERC20 - Avoid initializing variables to their default values

Description

When the aim is to initialize a variable to its default value, avoid initializing it explicitly as it costs more gas, especially when it is a storage variable. Simply declaring the variable is enough.

```
bool public isLiquidityPoolCreated = false; // @audit no need to set it to false as it is
false by default
```

Recommendation

Make the adequate changes.

Status

Acknowledged

[G-03] BaseERC20#swapTokensForETH - Checking `isSwapping` and `isLiquidityPoolCreated` is unnecessary

Description

`swapTokensForETH` can be rewritten in a more optimized way by removing some checks and some variables that are not very necessary.

Recommendation

Remove `isSwapping` completely as it is useless and `isLiquidityPoolCreated` as it is not really a necessary check

```
if (isSwapping || !isLiquidityPoolCreated) return;
```

Status

Acknowledged

[G-04] Curve - Constant `M` is not essential to avoid loss of precision and can be removed safely

Description

`M` is not necessary for better precision:

```
uint256 virtualNativeReserveAfter = virtualReserves[0] * virtualReserves[1] * Constants.M
/ Constants.VIRTUAL_TOKEN_RESERVE_WHEN_ADD_LIQUIDITY / Constants.M;
```

Recommendation

Remove `M` completely.

Status

Acknowledged

[G-05] BaseERC20 - Some custom errors remain unused and can be removed

Description

These 2 custom errors remain unused, either remove them to make the code size smaller or use them adequately, I believe the contract is safe without them, you might want to use `ErrMinimumInitialDeposit()`, it remains your choice, I think `ErrLiquidityReadyToAdd()` does not add much to the contract.

```
error ErrMinimumInitialDeposit(); // @audit never thrown

//...

error ErrLiquidityReadyToAdd(); // @audit never thrown
```

Recommendation

Remove `ErrLiquidityReadyToAdd()` completely and decide what you want to do about `ErrMinimumInitialDeposit()`.

Status

Acknowledged

QA

[Q-01] Layout of a solidity source file should be respected in the codebase

Description

The layout of a solidity source file should be respected in the codebase for better readability.

Recommendation

Check the layout of a solidity source file and make the adequate changes for better readability: <https://docs.soliditylang.org/en/latest/layout-of-source-files.html>

Status

Acknowledged

[Q-02] BaseERC20#buyExactOut - Wrong error thrown

Description

The error's name is not adequate with the scenario:

```
if (ethInputAmount > msg.value) {  
    revert ErrExceedsSlipage(); // @audit wrong error  
}
```

Recommendation

Change the error into something more adequate like ErrInsufficientFunds()

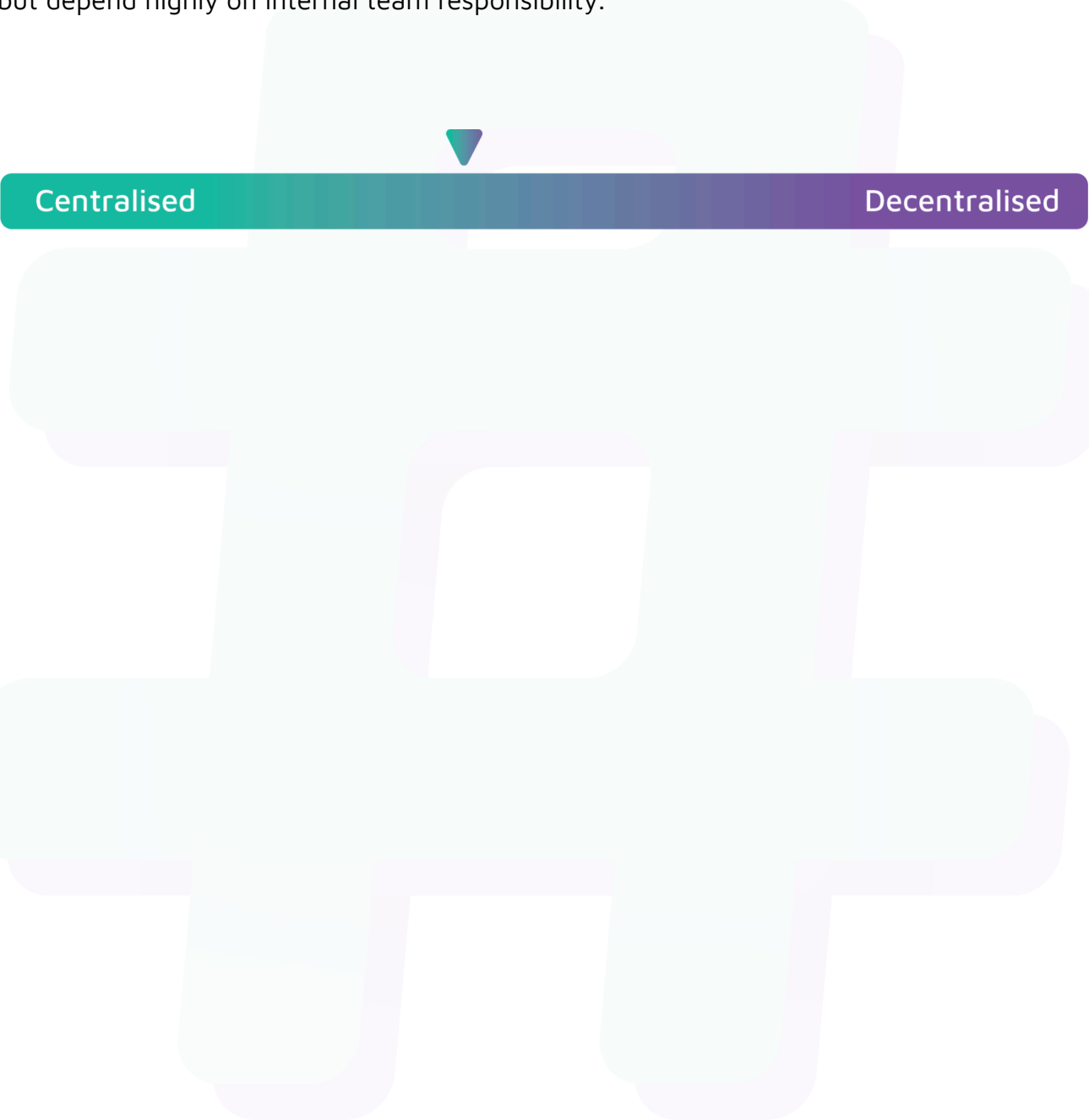
Status

Acknowledged

Centralisation

The OmniSwap project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the OmniSwap project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.